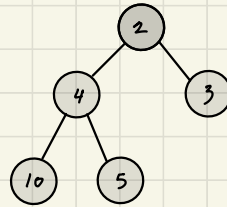



Priority Queues / Heaps

- PQ / Heaps: An ADT in which each element has both a value & a priority. The item with the highest priority is served first.
- Insertions are done freely
- Removals occur at min value or at value w/ max priority

Binary Heap

- Each node is less than or equal to its children.
- Is a complete binary tree.
 - All levels are fully filled except for the last
 - Guarantees $\Theta(\log n)$



0	1	2	3	4
2	4	3	10	5

$$i = 1$$

$$i_L = 3 \rightarrow (2 \cdot 1 + 1)$$

$$i_R = 4 \rightarrow (2 \cdot 1 + 2)$$

$$i_P = 0 \rightarrow (1 - 1) // 2$$

- Array implementation: For any node at index i ...
 - Left child is index $2i + 1$
 - Right child is index $2i + 2$
 - Parent $i = (i - 1) // 2$

Insertion:

1. Place new element in next available spot.
2. "Bubble up" (upheap) until heap-order property is restored

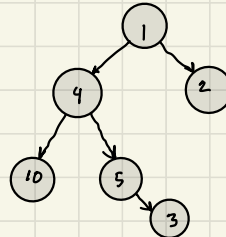
0	1	2	3	4	5
2	4	3	10	5	1

$$\rightarrow (5 - 1) // 2 = 2$$

0	1	2	3	4	5
2	4	1	10	5	3

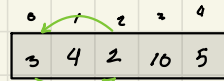
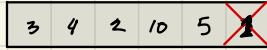
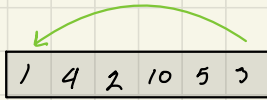
$$\rightarrow (2 - 1) // 2 = 0$$

1	4	2	10	5	3
---	---	---	----	---	---



• Removal

1. Swap root w/ last element
2. Remove the last element (former root/head)
3. If necessary, "bubble down" (downheap) the new root.



$$i_l = 2 * 0 + 1 = 1 \rightarrow 4$$

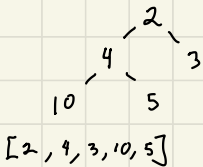
$$i_r = 2 * 0 + 2 = 2 \rightarrow 2$$



$$i_l = 2 * 2 + 1 = 5$$

$$i_r = 6$$

> outside of array



- Parent has higher priority than children
- Complete binary tree: all levels must be full except for maybe last one.

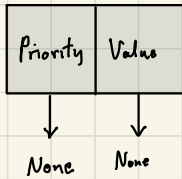
- Left child of j : $2j + 1$
- Right child of j : $2j + 2$
- Parent: $(j - 1) // 2$



```
def right(self, j):
    return 2 * j + 1
def left(self, j):
    return 2 * j
def parent(self, j):
    return j // 2
```

```
def has_left(self, j):
    return self.left(j) <= len(self.data) - 1
```

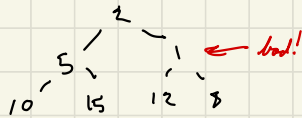
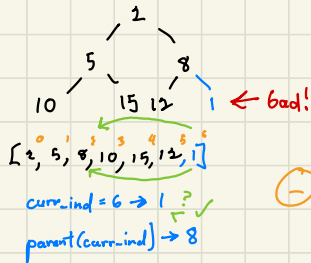
```
def has_right(self, j):
    return self.right(j) <= len(self.data) - 1
```



class Item:

```
def __init__(self, priority, value=None):
    self.priority = priority
    self.value = value
```

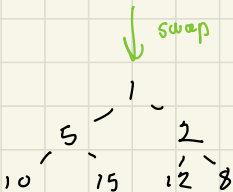
```
def __lt__(self, other):
    return self.priority <= other.priority
```



[2, 5, 1, 10, 15, 12, 8]

curr_ind = 2 -> 1

parent(2) -> 2 ✓



```
def fix_up(self, j):
```

```
    curr_ind = j
```

```
    keep_going = True
```

```
    while keep_going and curr_ind > 1:
```

```
        parent_ind = self.parent(curr_ind)
```

```
        if self.data[parent_ind] <= self.data[curr_ind]:
```

```
            keep_going = False
```

```
        else:
```

```
            self.swap(curr_ind, parent_ind)
```

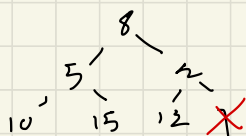
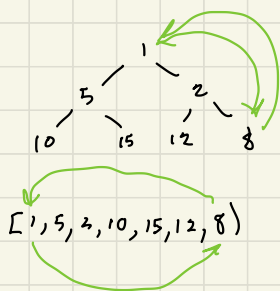
```
            curr_ind = parent_ind
```

```
def insert(self, priority, value=None):
```

```
    new_item = ArrayMinHeap.Item(priority, value)
```

```
    self.data.append(new_item)
```

```
    self.fix_up(len(data) - 1)
```



[8, 5, 2, 10, 15, 12, ~~8~~]

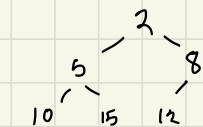
curr_ind = 0 → 8

left(0) = 1 → 5

right(0) = 2 → 2

smallest → 2

swap



[2, 5, 8, 10, 15, 12]

curr_ind = 2 → 8

left(2) → 12

right(2) → None

def delete_min(self):

if self.is_empty():

raise Exception()

self.swap(1, len(self.data) - 1)

item = self.data.pop()

if not self.is_empty():

self.fix_down(1)

return item

While the current node's priority > one of its children
swap w/ smaller child.

def fix_down(self, j):

curr_ind = j

keep_going = True

while keep_going and self.has_left(curr_ind):

left_ind = self.left(curr_ind)

right_ind = self.right(curr_ind)

small_child_ind = right_ind if self.data[right_ind] <= self.data[left_ind] else left_ind

if self.data[curr_ind] > self.data[small_child_ind]:

self.swap(curr_ind, small_child_ind)

curr_ind = small_child_ind

else:

keep_going = False

class ArrayMinHeap:

def __init__(self, priorities_lst=None, values_lst=None):

self.data = [None]

left: $2j+1 \rightarrow 2j$
right: $2j+2 \rightarrow 2j+1$
parent: $(j-1)//2 \rightarrow j//2$

if priorities_lst is not None:

for i in range(len(priorities_lst)):

new_item = ArrayMinHeap.Item(priorities_lst[i], values_lst[i])

self.data.append(new_item)

first_non_leaf_ind = self.parent(len(self.data) - 1)

for j in range(first_non_leaf_ind, 0, -1):

self.fix_down(j)

$$(0 \rightarrow h)_{\text{height}} = 1 + 2 + 4 + \dots + 2^h \approx 2^{h+1} - 1$$

$$h \approx \log_2(n)$$

$$\boxed{O(\log n)}$$